
Simple JWT Documentation

Release 5.2.2.post27+g47b7a08

David Sanders

Jul 06, 2023

1	Acknowledgments	3
2	Contents	5
2.1	Getting started	5
2.1.1	Requirements	5
2.1.2	Installation	5
2.1.3	Cryptographic Dependencies (Optional)	5
2.1.4	Project Configuration	6
2.1.5	Usage	6
2.2	Settings	7
2.2.1	ACCESS_TOKEN_LIFETIME	8
2.2.2	REFRESH_TOKEN_LIFETIME	8
2.2.3	ROTATE_REFRESH_TOKENS	9
2.2.4	BLACKLIST_AFTER_ROTATION	9
2.2.5	UPDATE_LAST_LOGIN	9
2.2.6	ALGORITHM	9
2.2.7	SIGNING_KEY	9
2.2.8	VERIFYING_KEY	9
2.2.9	AUDIENCE	10
2.2.10	ISSUER	10
2.2.11	JWK_URL	10
2.2.12	LEEWAY	10
2.2.13	AUTH_HEADER_TYPES	10
2.2.14	AUTH_HEADER_NAME	10
2.2.15	USER_ID_FIELD	10
2.2.16	USER_ID_CLAIM	11
2.2.17	USER_AUTHENTICATION_RULE	11
2.2.18	AUTH_TOKEN_CLASSES	11
2.2.19	TOKEN_TYPE_CLAIM	11
2.2.20	JTI_CLAIM	11
2.2.21	TOKEN_USER_CLASS	11
2.2.22	SLIDING_TOKEN_LIFETIME	11
2.2.23	SLIDING_TOKEN_REFRESH_LIFETIME	11
2.2.24	SLIDING_TOKEN_REFRESH_EXP_CLAIM	12
2.2.25	CHECK_REVOKE_TOKEN	12
2.2.26	REVOKE_TOKEN_CLAIM	12

2.3	Customizing token claims	12
2.4	Creating tokens manually	13
2.5	Token types	13
2.5.1	Sliding tokens	13
2.6	Blacklist app	14
2.7	Stateless User Authentication	15
2.7.1	JWTStatelessUserAuthentication backend	15
2.8	Development and contributing	15
2.9	drf-yasg Integration	16
2.10	rest_framework_simplejwt package	18
2.10.1	Submodules	18
2.10.2	rest_framework_simplejwt.authentication module	18
2.10.3	rest_framework_simplejwt.models module	19
2.10.4	rest_framework_simplejwt.serializers module	19
2.10.5	rest_framework_simplejwt.tokens module	21
2.10.6	rest_framework_simplejwt.utils module	22
2.10.7	rest_framework_simplejwt.views module	23
2.10.8	Module contents	24
3	Indices and tables	25
	Python Module Index	27
	Index	29

A JSON Web Token authentication plugin for the [Django REST Framework](#).

Simple JWT provides a JSON Web Token authentication backend for the Django REST Framework. It aims to cover the most common use cases of JWTs by offering a conservative set of default features. It also aims to be easily extensible in case a desired feature is not present.

CHAPTER 1

Acknowledgments

This project borrows code from the [Django REST Framework](#) as well as concepts from the implementation of another JSON web token library for the Django REST Framework, [django-rest-framework-jwt](#). The licenses from both of those projects have been included in this repository in the “licenses” directory.

2.1 Getting started

2.1.1 Requirements

- Python (3.8, 3.9, 3.10, 3.11)
- Django (3.2, 4.0, 4.1, 4.2)
- Django REST Framework (3.10, 3.11, 3.12, 3.13, 3.14)

These are the officially supported python and package versions. Other versions will probably work. You're free to modify the tox config and see what is possible.

2.1.2 Installation

Simple JWT can be installed with pip:

```
pip install djangorestframework-simplejwt
```

2.1.3 Cryptographic Dependencies (Optional)

If you are planning on encoding or decoding tokens using certain digital signature algorithms (i.e. RSA and ECDSA; visit [PyJWT](#) for other algorithms), you will need to install the [cryptography](#) library. This can be installed explicitly, or as a required extra in the `djangorestframework-simplejwt` requirement:

```
pip install djangorestframework-simplejwt[crypto]
```

The `djangorestframework-simplejwt[crypto]` format is recommended in requirements files in projects using Simple JWT, as a separate `cryptography` requirement line may later be mistaken for an unused requirement and removed.

2.1.4 Project Configuration

Then, your django project must be configured to use the library. In `settings.py`, add `rest_framework_simplejwt.authentication.JWTAuthentication` to the list of authentication classes:

```
REST_FRAMEWORK = {
    ...
    'DEFAULT_AUTHENTICATION_CLASSES': (
        ...
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    )
    ...
}
```

Also, in your root `urls.py` file (or any other url config), include routes for Simple JWT's `TokenObtainPairView` and `TokenRefreshView` views:

```
from rest_framework_simplejwt.views import (
    TokenObtainPairView,
    TokenRefreshView,
)

urlpatterns = [
    ...
    path('api/token/', TokenObtainPairView.as_view(), name='token_obtain_pair'),
    path('api/token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),
    ...
]
```

You can also include a route for Simple JWT's `TokenVerifyView` if you wish to allow API users to verify HMAC-signed tokens without having access to your signing key:

```
from rest_framework_simplejwt.views import TokenVerifyView

urlpatterns = [
    ...
    path('api/token/verify/', TokenVerifyView.as_view(), name='token_verify'),
    ...
]
```

If you wish to use localizations/translations, simply add `rest_framework_simplejwt` to `INSTALLED_APPS`.

```
INSTALLED_APPS = [
    ...
    'rest_framework_simplejwt',
    ...
]
```

2.1.5 Usage

To verify that Simple JWT is working, you can use `curl` to issue a couple of test requests:

```
curl \
  -X POST \
  -H "Content-Type: application/json" \
```

(continues on next page)

(continued from previous page)

```

"VERIFYING_KEY": "",
"AUDIENCE": None,
"ISSUER": None,
"JSON_ENCODER": None,
"JWK_URL": None,
"LEEWAY": 0,

"AUTH_HEADER_TYPES": ("Bearer",),
"AUTH_HEADER_NAME": "HTTP_AUTHORIZATION",
"USER_ID_FIELD": "id",
"USER_ID_CLAIM": "user_id",
"USER_AUTHENTICATION_RULE": "rest_framework_simplejwt.authentication.default_user_
↪authentication_rule",

"AUTH_TOKEN_CLASSES": ("rest_framework_simplejwt.tokens.AccessToken",),
"TOKEN_TYPE_CLAIM": "token_type",
"TOKEN_USER_CLASS": "rest_framework_simplejwt.models.TokenUser",

"JTI_CLAIM": "jti",

"SLIDING_TOKEN_REFRESH_EXP_CLAIM": "refresh_exp",
"SLIDING_TOKEN_LIFETIME": timedelta(minutes=5),
"SLIDING_TOKEN_REFRESH_LIFETIME": timedelta(days=1),

"TOKEN_OBTAIN_SERIALIZER": "rest_framework_simplejwt.serializers.
↪TokenObtainPairSerializer",
"TOKEN_REFRESH_SERIALIZER": "rest_framework_simplejwt.serializers.
↪TokenRefreshSerializer",
"TOKEN_VERIFY_SERIALIZER": "rest_framework_simplejwt.serializers.
↪TokenVerifySerializer",
"TOKEN_BLACKLIST_SERIALIZER": "rest_framework_simplejwt.serializers.
↪TokenBlacklistSerializer",
"SLIDING_TOKEN_OBTAIN_SERIALIZER": "rest_framework_simplejwt.serializers.
↪TokenObtainSlidingSerializer",
"SLIDING_TOKEN_REFRESH_SERIALIZER": "rest_framework_simplejwt.serializers.
↪TokenRefreshSlidingSerializer",
}

```

Above, the default values for these settings are shown.

2.2.1 ACCESS_TOKEN_LIFETIME

A `datetime.timedelta` object which specifies how long access tokens are valid. This `timedelta` value is added to the current UTC time during token generation to obtain the token's default "exp" claim value.

2.2.2 REFRESH_TOKEN_LIFETIME

A `datetime.timedelta` object which specifies how long refresh tokens are valid. This `timedelta` value is added to the current UTC time during token generation to obtain the token's default "exp" claim value.

2.2.3 ROTATE_REFRESH_TOKENS

When set to `True`, if a refresh token is submitted to the `TokenRefreshView`, a new refresh token will be returned along with the new access token. This new refresh token will be supplied via a “refresh” key in the JSON response. New refresh tokens will have a renewed expiration time which is determined by adding the `timedelta` in the `REFRESH_TOKEN_LIFETIME` setting to the current time when the request is made. If the blacklist app is in use and the `BLACKLIST_AFTER_ROTATION` setting is set to `True`, refresh tokens submitted to the refresh view will be added to the blacklist.

2.2.4 BLACKLIST_AFTER_ROTATION

When set to `True`, causes refresh tokens submitted to the `TokenRefreshView` to be added to the blacklist if the blacklist app is in use and the `ROTATE_REFRESH_TOKENS` setting is set to `True`. You need to add `'rest_framework_simplejwt.token_blacklist'`, to your `INSTALLED_APPS` in the settings file to use this setting.

Learn more about [Blacklist app](#).

2.2.5 UPDATE_LAST_LOGIN

When set to `True`, `last_login` field in the `auth_user` table is updated upon login (`TokenObtainPairView`).

Warning: Updating `last_login` will dramatically increase the number of database transactions. People abusing the views could slow the server and this could be a security vulnerability. If you really want this, throttle the endpoint with DRF at the very least.

2.2.6 ALGORITHM

The algorithm from the PyJWT library which will be used to perform signing/verification operations on tokens. To use symmetric HMAC signing and verification, the following algorithms may be used: `'HS256'`, `'HS384'`, `'HS512'`. When an HMAC algorithm is chosen, the `SIGNING_KEY` setting will be used as both the signing key and the verifying key. In that case, the `VERIFYING_KEY` setting will be ignored. To use asymmetric RSA signing and verification, the following algorithms may be used: `'RS256'`, `'RS384'`, `'RS512'`. When an RSA algorithm is chosen, the `SIGNING_KEY` setting must be set to a string that contains an RSA private key. Likewise, the `VERIFYING_KEY` setting must be set to a string that contains an RSA public key.

2.2.7 SIGNING_KEY

The signing key that is used to sign the content of generated tokens. For HMAC signing, this should be a random string with at least as many bits of data as is required by the signing protocol. For RSA signing, this should be a string that contains an RSA private key that is 2048 bits or longer. Since Simple JWT defaults to using 256-bit HMAC signing, the `SIGNING_KEY` setting defaults to the value of the `SECRET_KEY` setting for your django project. Although this is the most reasonable default that Simple JWT can provide, it is recommended that developers change this setting to a value that is independent from the django project secret key. This will make changing the signing key used for tokens easier in the event that it is compromised.

2.2.8 VERIFYING_KEY

The verifying key which is used to verify the content of generated tokens. If an HMAC algorithm has been specified by the `ALGORITHM` setting, the `VERIFYING_KEY` setting will be ignored and the value of the `SIGNING_KEY` setting

will be used. If an RSA algorithm has been specified by the `ALGORITHM` setting, the `VERIFYING_KEY` setting must be set to a string that contains an RSA public key.

2.2.9 AUDIENCE

The audience claim to be included in generated tokens and/or validated in decoded tokens. When set to `None`, this field is excluded from tokens and is not validated.

2.2.10 ISSUER

The issuer claim to be included in generated tokens and/or validated in decoded tokens. When set to `None`, this field is excluded from tokens and is not validated.

2.2.11 JWK_URL

The `JWK_URL` is used to dynamically resolve the public keys needed to verify the signing of tokens. When using Auth0 for example you might set this to `'https://yourdomain.auth0.com/.well-known/jwks.json'`. When set to `None`, this field is excluded from the token backend and is not used during validation.

2.2.12 LEEWAY

Leeway is used to give some margin to the expiration time. This can be an integer for seconds or a `datetime.timedelta`. Please reference <https://pyjwt.readthedocs.io/en/latest/usage.html#expiration-time-claim-exp> for more information.

2.2.13 AUTH_HEADER_TYPES

The authorization header type(s) that will be accepted for views that require authentication. For example, a value of `'Bearer'` means that views requiring authentication would look for a header with the following format: `Authorization: Bearer <token>`. This setting may also contain a list or tuple of possible header types (e.g. `('Bearer', 'JWT')`). If a list or tuple is used in this way, and authentication fails, the first item in the collection will be used to build the “WWW-Authenticate” header in the response.

2.2.14 AUTH_HEADER_NAME

The authorization header name to be used for authentication. The default is `HTTP_AUTHORIZATION` which will accept the `Authorization` header in the request. For example if you'd like to use `X_Access-Token` in the header of your requests please specify the `AUTH_HEADER_NAME` to be `HTTP_X_ACCESS_TOKEN` in your settings.

2.2.15 USER_ID_FIELD

The database field from the user model that will be included in generated tokens to identify users. It is recommended that the value of this setting specifies a field that does not normally change once its initial value is chosen. For example, specifying a “username” or “email” field would be a poor choice since an account’s username or email might change depending on how account management in a given service is designed. This could allow a new account to be created with an old username while an existing token is still valid which uses that username as a user identifier.

2.2.16 USER_ID_CLAIM

The claim in generated tokens which will be used to store user identifiers. For example, a setting value of `'user_id'` would mean generated tokens include a “user_id” claim that contains the user’s identifier.

2.2.17 USER_AUTHENTICATION_RULE

Callable to determine if the user is permitted to authenticate. This rule is applied after a valid token is processed. The user object is passed to the callable as an argument. The default rule is to check that the `is_active` flag is still `True`. The callable must return a boolean, `True` if authorized, `False` otherwise resulting in a 401 status code.

2.2.18 AUTH_TOKEN_CLASSES

A list of dot paths to classes that specify the types of token that are allowed to prove authentication. More about this in the “Token types” section below.

2.2.19 TOKEN_TYPE_CLAIM

The claim name that is used to store a token’s type. More about this in the “Token types” section below.

2.2.20 JTI_CLAIM

The claim name that is used to store a token’s unique identifier. This identifier is used to identify revoked tokens in the blacklist app. It may be necessary in some cases to use another claim besides the default “jti” claim to store such a value.

2.2.21 TOKEN_USER_CLASS

A stateless user object which is backed by a validated token. Used only for the `JWTStatelessUserAuthentication` authentication backend. The value is a dotted path to your subclass of `rest_framework_simplejwt.models.TokenUser`, which also is the default.

2.2.22 SLIDING_TOKEN_LIFETIME

A `datetime.timedelta` object which specifies how long sliding tokens are valid to prove authentication. This `timedelta` value is added to the current UTC time during token generation to obtain the token’s default “exp” claim value. More about this in the “Sliding tokens” section below.

2.2.23 SLIDING_TOKEN_REFRESH_LIFETIME

A `datetime.timedelta` object which specifies how long sliding tokens are valid to be refreshed. This `timedelta` value is added to the current UTC time during token generation to obtain the token’s default “exp” claim value. More about this in the “Sliding tokens” section below.

2.2.24 SLIDING_TOKEN_REFRESH_EXP_CLAIM

The claim name that is used to store the expiration time of a sliding token's refresh period. More about this in the "Sliding tokens" section below.

2.2.25 CHECK_REVOKE_TOKEN

If this field is set to `True`, the system will verify whether the token has been revoked or not by comparing the md5 hash of the user's current password with the value stored in the `REVOKE_TOKEN_CLAIM` field within the payload of the JWT token.

2.2.26 REVOKE_TOKEN_CLAIM

The claim name that is used to store a user hash password. If the value of this `CHECK_REVOKE_TOKEN` field is `True`, this field will be included in the JWT payload.

2.3 Customizing token claims

If you wish to customize the claims contained in web tokens which are generated by the `TokenObtainPairView` and `TokenObtainSlidingView` views, create a subclass for the desired view as well as a subclass for its corresponding serializer. Here's an example of how to customize the claims in tokens generated by the `TokenObtainPairView`:

```
from rest_framework_simplejwt.serializers import TokenObtainPairSerializer
from rest_framework_simplejwt.views import TokenObtainPairView

class MyTokenObtainPairSerializer(TokenObtainPairSerializer):
    @classmethod
    def get_token(cls, user):
        token = super().get_token(user)

        # Add custom claims
        token['name'] = user.name
        # ...

        return token
```

```
# Django project settings.py
...

SIMPLE_JWT = {
    # It will work instead of the default serializer(TokenObtainPairSerializer).
    "TOKEN_OBTAIN_SERIALIZER": "my_app.serializers.MyTokenObtainPairSerializer",
    # ...
}
```

Note that the example above will cause the customized claims to be present in both refresh *and* access tokens which are generated by the view. This follows from the fact that the `get_token` method above produces the *refresh* token for the view, which is in turn used to generate the view's access token.

As with the standard token views, you'll also need to include a url route to your subclassed view.

2.4 Creating tokens manually

Sometimes, you may wish to manually create a token for a user. This could be done as follows:

```
from rest_framework_simplejwt.tokens import RefreshToken

def get_tokens_for_user(user):
    refresh = RefreshToken.for_user(user)

    return {
        'refresh': str(refresh),
        'access': str(refresh.access_token),
    }
```

The above function `get_tokens_for_user` will return the serialized representations of new refresh and access tokens for the given user. In general, a token for any subclass of `rest_framework_simplejwt.tokens.Token` can be created in this way.

2.5 Token types

Simple JWT provides two different token types that can be used to prove authentication. In a token's payload, its type can be identified by the value of its token type claim, which is "token_type" by default. This may have a value of "access", "sliding", or "refresh" however refresh tokens are not considered valid for authentication at this time. The claim name used to store the type can be customized by changing the `TOKEN_TYPE_CLAIM` setting.

By default, Simple JWT expects an "access" token to prove authentication. The allowed auth token types are determined by the value of the `AUTH_TOKEN_CLASSES` setting. This setting contains a list of dot paths to token classes. It includes the `'rest_framework_simplejwt.tokens.AccessToken'` dot path by default but may also include the `'rest_framework_simplejwt.tokens.SlidingToken'` dot path. Either or both of those dot paths may be present in the list of auth token classes. If they are both present, then both of those token types may be used to prove authentication.

2.5.1 Sliding tokens

Sliding tokens offer a more convenient experience to users of tokens with the trade-offs of being less secure and, in the case that the blacklist app is being used, less performant. A sliding token is one which contains both an expiration claim and a refresh expiration claim. As long as the timestamp in a sliding token's expiration claim has not passed, it can be used to prove authentication. Additionally, as long as the timestamp in its refresh expiration claim has not passed, it may also be submitted to a refresh view to get another copy of itself with a renewed expiration claim.

If you want to use sliding tokens, change the `AUTH_TOKEN_CLASSES` setting to `('rest_framework_simplejwt.tokens.SlidingToken',)`. (Alternatively, the `AUTH_TOKEN_CLASSES` setting may include dot paths to both the `AccessToken` and `SlidingToken` token classes in the `rest_framework_simplejwt.tokens` module if you want to allow both token types to be used for authentication.)

Also, include urls for the sliding token specific `TokenObtainSlidingView` and `TokenRefreshSlidingView` views alongside or in place of urls for the access token specific `TokenObtainPairView` and `TokenRefreshView` views:

```
from rest_framework_simplejwt.views import (
    TokenObtainSlidingView,
    TokenRefreshSlidingView,
```

(continues on next page)

(continued from previous page)

```
)

urlpatterns = [
    ...
    path('api/token/', TokenObtainSlidingView.as_view(), name='token_obtain'),
    path('api/token/refresh/', TokenRefreshSlidingView.as_view(), name='token_refresh
↪'),
    ...
]
```

Be aware that, if you are using the blacklist app, Simple JWT will validate all sliding tokens against the blacklist for each authenticated request. This will reduce the performance of authenticated API views.

2.6 Blacklist app

Simple JWT includes an app that provides token blacklist functionality. To use this app, include it in your list of installed apps in `settings.py`:

```
# Django project settings.py

...

INSTALLED_APPS = (
    ...
    'rest_framework_simplejwt.token_blacklist',
    ...
)
```

Also, make sure to run `python manage.py migrate` to run the app's migrations.

If the blacklist app is detected in `INSTALLED_APPS`, Simple JWT will add any generated refresh or sliding tokens to a list of outstanding tokens. It will also check that any refresh or sliding token does not appear in a blacklist of tokens before it considers it as valid.

The Simple JWT blacklist app implements its outstanding and blacklisted token lists using two models: `OutstandingToken` and `BlacklistedToken`. Model admins are defined for both of these models. To add a token to the blacklist, find its corresponding `OutstandingToken` record in the admin and use the admin again to create a `BlacklistedToken` record that points to the `OutstandingToken` record.

Alternatively, you can blacklist a token by creating a `BlacklistMixin` subclass instance and calling the instance's `blacklist` method:

```
from rest_framework_simplejwt.tokens import RefreshToken

token = RefreshToken(base64_encoded_token_string)
token.blacklist()
```

This will create unique outstanding token and blacklist records for the token's "jti" claim or whichever claim is specified by the `JTI_CLAIM` setting.

In a `urls.py` file, you can also include a route for `TokenBlacklistView`:

```
from rest_framework_simplejwt.views import TokenBlacklistView

urlpatterns = [
```

(continues on next page)

If you're running a Mac and/or with zsh, you need to escape the brackets:

```
pip install -e .\[dev\]
```

To run the tests:

```
pytest
```

To run the tests in all supported environments with tox, first [install pyenv](#). Next, install the relevant Python minor versions and create a `.python-version` file in the project directory:

```
pyenv install 3.9.x
pyenv install 3.8.x
cat > .python-version <<EOF
3.9.x
3.8.x
EOF
```

Above, the `x` in each case should be replaced with the latest corresponding patch version. The `.python-version` file will tell pyenv and tox that you're testing against multiple versions of Python. Next, run tox:

```
tox
```

2.9 drf-yasg Integration

`drf-yasg` is a library that automatically generates an OpenAPI schema by inspecting DRF `Serializer` definitions. Because `django-rest-framework-simplejwt` serializers are not symmetric, if you want to generate correct OpenAPI schemas for your JWT token endpoints, use the following code to decorate your JWT View definitions.

```
from drf_yasg.utils import swagger_auto_schema
from rest_framework import serializers, status
from rest_framework_simplejwt.views import (
    TokenBlacklistView,
    TokenObtainPairView,
    TokenRefreshView,
    TokenVerifyView,
)

class TokenObtainPairResponseSerializer(serializers.Serializer):
    access = serializers.CharField()
    refresh = serializers.CharField()

    def create(self, validated_data):
        raise NotImplementedError()

    def update(self, instance, validated_data):
        raise NotImplementedError()

class DecoratedTokenObtainPairView(TokenObtainPairView):
    @swagger_auto_schema(
        responses={
            status.HTTP_200_OK: TokenObtainPairResponseSerializer,
        }
    )
```

(continues on next page)

(continued from previous page)

```

    )
    def post(self, request, *args, **kwargs):
        return super().post(request, *args, **kwargs)

class TokenRefreshResponseSerializer(serializers.Serializer):
    access = serializers.CharField()

    def create(self, validated_data):
        raise NotImplementedError()

    def update(self, instance, validated_data):
        raise NotImplementedError()

class DecoratedTokenRefreshView(TokenRefreshView):
    @swagger_auto_schema(
        responses={
            status.HTTP_200_OK: TokenRefreshResponseSerializer,
        }
    )
    def post(self, request, *args, **kwargs):
        return super().post(request, *args, **kwargs)

class TokenVerifyResponseSerializer(serializers.Serializer):
    def create(self, validated_data):
        raise NotImplementedError()

    def update(self, instance, validated_data):
        raise NotImplementedError()

class DecoratedTokenVerifyView(TokenVerifyView):
    @swagger_auto_schema(
        responses={
            status.HTTP_200_OK: TokenVerifyResponseSerializer,
        }
    )
    def post(self, request, *args, **kwargs):
        return super().post(request, *args, **kwargs)

class TokenBlacklistResponseSerializer(serializers.Serializer):
    def create(self, validated_data):
        raise NotImplementedError()

    def update(self, instance, validated_data):
        raise NotImplementedError()

class DecoratedTokenBlacklistView(TokenBlacklistView):
    @swagger_auto_schema(
        responses={
            status.HTTP_200_OK: TokenBlacklistResponseSerializer,
        }
    )

```

(continues on next page)

(continued from previous page)

```
def post(self, request, *args, **kwargs):
    return super().post(request, *args, **kwargs)
```

2.10 rest_framework_simplejwt package

2.10.1 Submodules

2.10.2 rest_framework_simplejwt.authentication module

```
class rest_framework_simplejwt.authentication.JWTAuthentication(*args,
                                                                **kwargs)
    Bases: rest_framework.authentication.BaseAuthentication
    An authentication plugin that authenticates requests through a JSON web token provided in a request header.
    authenticate(request: rest_framework.request.Request) → Optional[Tuple[AuthUser,
        rest_framework_simplejwt.tokens.Token]]
        Authenticate the request and return a two-tuple of (user, token).
    authenticate_header(request: rest_framework.request.Request) → str
        Return a string to be used as the value of the WWW-Authenticate header in a 401 Unauthenticated response,
        or None if the authentication scheme should return 403 Permission Denied responses.
    get_header(request: rest_framework.request.Request) → bytes
        Extracts the header containing the JSON web token from the given request.
    get_raw_token(header: bytes) → Optional[bytes]
        Extracts an unvalidated JSON web token from the given "Authorization" header value.
    get_user(validated_token: rest_framework_simplejwt.tokens.Token) → AuthUser
        Attempts to find and return a user using the given validated token.
    get_validated_token(raw_token: bytes) → rest_framework_simplejwt.tokens.Token
        Validates an encoded JSON web token and returns a validated token wrapper object.
    media_type = 'application/json'
    www_authenticate_realm = 'api'

class rest_framework_simplejwt.authentication.JWTStatelessUserAuthentication(*args,
                                                                              **kwargs)
    Bases: rest_framework_simplejwt.authentication.JWTAuthentication
    An authentication plugin that authenticates requests through a JSON web token provided in a request header
    without performing a database lookup to obtain a user instance.
    get_user(validated_token: rest_framework_simplejwt.tokens.Token) → AuthUser
        Returns a stateless user object which is backed by the given validated token.

rest_framework_simplejwt.authentication.JWTTokenUserAuthentication
    alias of rest_framework_simplejwt.authentication.JWTStatelessUserAuthentication

rest_framework_simplejwt.authentication.default_user_authentication_rule(user:
                                                                              AuthUser)
                                                                              →
                                                                              bool
```

2.10.3 rest_framework_simplejwt.models module

class rest_framework_simplejwt.models.**TokenUser** (*token: Token*)

Bases: `object`

A dummy user class modeled after `django.contrib.auth.models.AnonymousUser`. Used in conjunction with the *JWTStatelessUserAuthentication* backend to implement single sign-on functionality across services which share the same secret key. *JWTStatelessUserAuthentication* will return an instance of this class instead of a *User* model instance. Instances of this class act as stateless user objects which are backed by validated tokens.

check_password (*raw_password: str*) → None

delete () → None

get_all_permissions (*obj: Optional[object] = None*) → set

get_group_permissions (*obj: Optional[object] = None*) → set

get_username () → str

groups

has_module_perms (*module: str*) → bool

has_perm (*perm: str, obj: Optional[object] = None*) → bool

has_perms (*perm_list: List[str], obj: Optional[object] = None*) → bool

id

is_active = True

is_anonymous

is_authenticated

is_staff

is_superuser

pk

save () → None

set_password (*raw_password: str*) → None

user_permissions

username

2.10.4 rest_framework_simplejwt.serializers module

class rest_framework_simplejwt.serializers.**PasswordField** (**args, **kwargs*)

Bases: `rest_framework.fields.CharField`

class rest_framework_simplejwt.serializers.**TokenBlacklistSerializer** (*instance=None,*

*data=<class
'rest_framework.fields.empty'>,
**kwargs*)

Bases: `rest_framework.serializers.Serializer`

token_class

alias of `rest_framework_simplejwt.tokens.RefreshToken`

validate (*attrs: Dict[str, Any]*) → Dict[Any, Any]

```
class rest_framework_simplejwt.serializers.TokenObtainPairSerializer(*args,
                                                                    **kwargs)
    Bases: rest_framework_simplejwt.serializers.TokenObtainSerializer
    token_class
        alias of rest_framework_simplejwt.tokens.RefreshToken
    validate(attrs: Dict[str, Any]) → Dict[str, str]

class rest_framework_simplejwt.serializers.TokenObtainSerializer(*args,
                                                                    **kwargs)
    Bases: rest_framework.serializers.Serializer
    default_error_messages = {'no_active_account': 'No active account found with the given credentials'}
    classmethod get_token(user: AuthUser) → rest_framework_simplejwt.tokens.Token
    token_class = None
    username_field = 'username'
    validate(attrs: Dict[str, Any]) → Dict[Any, Any]

class rest_framework_simplejwt.serializers.TokenObtainSlidingSerializer(*args,
                                                                         **kwargs)
    Bases: rest_framework_simplejwt.serializers.TokenObtainSerializer
    token_class
        alias of rest_framework_simplejwt.tokens.SlidingToken
    validate(attrs: Dict[str, Any]) → Dict[str, str]

class rest_framework_simplejwt.serializers.TokenRefreshSerializer(instance=None,
                                                                    data=<class
                                                                    'rest_framework.fields.empty'>,
                                                                    **kwargs)
    Bases: rest_framework.serializers.Serializer
    token_class
        alias of rest_framework_simplejwt.tokens.RefreshToken
    validate(attrs: Dict[str, Any]) → Dict[str, str]

class rest_framework_simplejwt.serializers.TokenRefreshSlidingSerializer(instance=None,
                                                                           data=<class
                                                                           'rest_framework.fields.empty'>,
                                                                           **kwargs)
    Bases: rest_framework.serializers.Serializer
    token_class
        alias of rest_framework_simplejwt.tokens.SlidingToken
    validate(attrs: Dict[str, Any]) → Dict[str, str]

class rest_framework_simplejwt.serializers.TokenVerifySerializer(instance=None,
                                                                    data=<class
                                                                    'rest_framework.fields.empty'>,
                                                                    **kwargs)
    Bases: rest_framework.serializers.Serializer
    validate(attrs: Dict[str, None]) → Dict[Any, Any]
```


2.10.5 rest_framework_simplejwt.tokens module

```
class rest_framework_simplejwt.tokens.AccessToken(token: Optional[Token] = None,
                                                  verify: bool = True)
    Bases: rest_framework_simplejwt.tokens.Token
    lifetime = datetime.timedelta(seconds=300)
    token_type = 'access'

class rest_framework_simplejwt.tokens.BlacklistMixin
    Bases: object

    If the rest_framework_simplejwt.token_blacklist app was configured to be used, tokens created from Blacklist-Mixin subclasses will insert themselves into an outstanding token list and also check for their membership in a token blacklist.

    blacklist() → rest_framework_simplejwt.token_blacklist.models.BlacklistedToken
        Ensures this token is included in the outstanding token list and adds it to the blacklist.

    check_blacklist() → None
        Checks if this token is present in the token blacklist. Raises TokenError if so.

    classmethod for_user(user: AuthUser) → rest_framework_simplejwt.tokens.Token
        Adds this token to the outstanding token list.

    verify(*args, **kwargs) → None

class rest_framework_simplejwt.tokens.RefreshToken(token: Optional[Token] = None,
                                                  verify: bool = True)
    Bases: rest_framework_simplejwt.tokens.BlacklistMixin,
           rest_framework_simplejwt.tokens.Token
    access_token
        Returns an access token created from this refresh token. Copies all claims present in this refresh token to the new access token except those claims listed in the no_copy_claims attribute.
    access_token_class
        alias of AccessToken
    lifetime = datetime.timedelta(days=1)
    no_copy_claims = ('token_type', 'exp', 'jti', 'jti')
    token_type = 'refresh'

class rest_framework_simplejwt.tokens.SlidingToken(*args, **kwargs)
    Bases: rest_framework_simplejwt.tokens.BlacklistMixin,
           rest_framework_simplejwt.tokens.Token
    lifetime = datetime.timedelta(seconds=300)
    token_type = 'sliding'

class rest_framework_simplejwt.tokens.Token(token: Optional[Token] = None, verify: bool = True)
    Bases: object

    A class which validates and wraps an existing JWT or can be used to build a new JWT.

    check_exp(claim: str = 'exp', current_time: Optional[datetime.datetime] = None) → None
        Checks whether a timestamp value in the given claim has passed (since the given datetime value in current_time). Raises a TokenError with a user-facing error message if so.
```

classmethod for_user (*user: AuthUser*) → *rest_framework_simplejwt.tokens.Token*

Returns an authorization token for the given user that will be provided after authenticating the user's credentials.

get (*key: str, default: Optional[Any] = None*) → *Any*

get_token_backend () → *TokenBackend*

lifetime = *None*

set_exp (*claim: str = 'exp', from_time: Optional[datetime.datetime] = None, lifetime: Optional[datetime.timedelta] = None*) → *None*

Updates the expiration time of a token.

See here: <https://tools.ietf.org/html/rfc7519#section-4.1.4>

set_iat (*claim: str = 'iat', at_time: Optional[datetime.datetime] = None*) → *None*

Updates the time at which the token was issued.

See here: <https://tools.ietf.org/html/rfc7519#section-4.1.6>

set_jti () → *None*

Populates the configured jti claim of a token with a string where there is a negligible probability that the same string will be chosen at a later time.

See here: <https://tools.ietf.org/html/rfc7519#section-4.1.7>

token_backend

token_type = *None*

verify () → *None*

Performs additional validation steps which were not performed when this token was decoded. This method is part of the “public” API to indicate the intention that it may be overridden in subclasses.

verify_token_type () → *None*

Ensures that the token type claim is present and has the correct value.

class *rest_framework_simplejwt.tokens.UntypedToken* (*token: Optional[Token] = None, verify: bool = True*)

Bases: *rest_framework_simplejwt.tokens.Token*

lifetime = *datetime.timedelta(0)*

token_type = *'untyped'*

verify_token_type () → *None*

Untyped tokens do not verify the “token_type” claim. This is useful when performing general validation of a token's signature and other properties which do not relate to the token's intended use.

2.10.6 rest_framework_simplejwt.utils module

rest_framework_simplejwt.utils.aware_utcnow () → *datetime.datetime*

rest_framework_simplejwt.utils.datetime_from_epoch (*ts: float*) → *datetime.datetime*

rest_framework_simplejwt.utils.datetime_to_epoch (*dt: datetime.datetime*) → *int*

rest_framework_simplejwt.utils.format_lazy (*s: str, *args, **kwargs*) → *str*

rest_framework_simplejwt.utils.get_md5_hash_password (*password: str*) → *str*

Returns MD5 hash of the given password

rest_framework_simplejwt.utils.make_utc (*dt: datetime.datetime*) → *datetime.datetime*

2.10.7 rest_framework_simplejwt.views module

class `rest_framework_simplejwt.views.TokenBlacklistView` (**kwargs)

Bases: `rest_framework_simplejwt.views.TokenViewBase`

Takes a token and blacklists it. Must be used with the `rest_framework_simplejwt.token_blacklist` app installed.

class `rest_framework_simplejwt.views.TokenObtainPairView` (**kwargs)

Bases: `rest_framework_simplejwt.views.TokenViewBase`

Takes a set of user credentials and returns an access and refresh JSON web token pair to prove the authentication of those credentials.

class `rest_framework_simplejwt.views.TokenObtainSlidingView` (**kwargs)

Bases: `rest_framework_simplejwt.views.TokenViewBase`

Takes a set of user credentials and returns a sliding JSON web token to prove the authentication of those credentials.

class `rest_framework_simplejwt.views.TokenRefreshSlidingView` (**kwargs)

Bases: `rest_framework_simplejwt.views.TokenViewBase`

Takes a sliding JSON web token and returns a new, refreshed version if the token's refresh period has not expired.

class `rest_framework_simplejwt.views.TokenRefreshView` (**kwargs)

Bases: `rest_framework_simplejwt.views.TokenViewBase`

Takes a refresh type JSON web token and returns an access type JSON web token if the refresh token is valid.

class `rest_framework_simplejwt.views.TokenVerifyView` (**kwargs)

Bases: `rest_framework_simplejwt.views.TokenViewBase`

Takes a token and indicates if it is valid. This view provides no information about a token's fitness for a particular use.

class `rest_framework_simplejwt.views.TokenViewBase` (**kwargs)

Bases: `rest_framework.generics.GenericAPIView`

authentication_classes = ()

get_authenticate_header (*request*: `rest_framework.request.Request`) → str

If a request is unauthenticated, determine the WWW-Authenticate header to use for 401 responses, if any.

get_serializer_class () → `rest_framework.serializers.Serializer`

If `serializer_class` is set, use it directly. Otherwise get the class from settings.

permission_classes = ()

post (*request*: `rest_framework.request.Request`, *args, **kwargs) → `rest_framework.response.Response`

serializer_class = None

www_authenticate_realm = 'api'

`rest_framework_simplejwt.views.token_blacklist` (*request*, *args, **kwargs)

Takes a token and blacklists it. Must be used with the `rest_framework_simplejwt.token_blacklist` app installed.

`rest_framework_simplejwt.views.token_obtain_pair` (*request*, *args, **kwargs)

Takes a set of user credentials and returns an access and refresh JSON web token pair to prove the authentication of those credentials.

`rest_framework_simplejwt.views.token_obtain_sliding(request, *args, **kwargs)`
Takes a set of user credentials and returns a sliding JSON web token to prove the authentication of those credentials.

`rest_framework_simplejwt.views.token_refresh(request, *args, **kwargs)`
Takes a refresh type JSON web token and returns an access type JSON web token if the refresh token is valid.

`rest_framework_simplejwt.views.token_refresh_sliding(request, *args, **kwargs)`
Takes a sliding JSON web token and returns a new, refreshed version if the token's refresh period has not expired.

`rest_framework_simplejwt.views.token_verify(request, *args, **kwargs)`
Takes a token and indicates if it is valid. This view provides no information about a token's fitness for a particular use.

2.10.8 Module contents

CHAPTER 3

Indices and tables

- `genindex`
- `modindex`

r

- `rest_framework_simplejwt`, [24](#)
- `rest_framework_simplejwt.authentication`,
[18](#)
- `rest_framework_simplejwt.models`, [19](#)
- `rest_framework_simplejwt.serializers`,
[19](#)
- `rest_framework_simplejwt.tokens`, [21](#)
- `rest_framework_simplejwt.utils`, [22](#)
- `rest_framework_simplejwt.views`, [23](#)

A

`access_token` (*rest_framework_simplejwt.tokens.RefreshToken* attribute), 21
`access_token_class` (*rest_framework_simplejwt.tokens.RefreshToken* attribute), 21
`AccessToken` (class in *rest_framework_simplejwt.tokens*), 21
`authenticate()` (*rest_framework_simplejwt.authentication.JWTAuthentication* method), 18
`authenticate_header()` (*rest_framework_simplejwt.authentication.JWTAuthentication* method), 18
`authentication_classes` (*rest_framework_simplejwt.views.TokenViewBase* attribute), 23
`aware_utcnow()` (in module *rest_framework_simplejwt.utils*), 22

B

`blacklist()` (*rest_framework_simplejwt.tokens.BlacklistMixin* method), 21
`BlacklistMixin` (class in *rest_framework_simplejwt.tokens*), 21

C

`check_blacklist()` (*rest_framework_simplejwt.tokens.BlacklistMixin* method), 21
`check_exp()` (*rest_framework_simplejwt.tokens.Token* method), 21
`check_password()` (*rest_framework_simplejwt.models.TokenUser* method), 19

D

`datetime_from_epoch()` (in module *rest_framework_simplejwt.utils*), 22
`datetime_to_epoch()` (in module *rest_framework_simplejwt.utils*), 22

`default_error_messages`

`default_user_authentication_rule()` (in module *rest_framework_simplejwt.authentication*), 18

`delete()` (*rest_framework_simplejwt.models.TokenUser* method), 19

`for_user()` (*rest_framework_simplejwt.tokens.BlacklistMixin* class method), 21

`format_lazy()` (in module *rest_framework_simplejwt.utils*), 22

G

`get()` (*rest_framework_simplejwt.tokens.Token* method), 22
`get_all_permissions()` (*rest_framework_simplejwt.models.TokenUser* method), 19

`get_authenticate_header()` (*rest_framework_simplejwt.views.TokenViewBase* method), 23
`get_group_permissions()` (*rest_framework_simplejwt.models.TokenUser* method), 19
`get_header()` (*rest_framework_simplejwt.authentication.JWTAuthentication* method), 18

`get_md5_hash_password()` (in module *rest_framework_simplejwt.utils*), 22
`get_raw_token()` (*rest_framework_simplejwt.authentication.JWTAuthentication* method), 18

`get_serializer_class()` (*rest_framework_simplejwt.views.TokenViewBase* method), 23

`get_token()` (*rest_framework_simplejwt.serializers.TokenObtainSerializer* class method), 20

[illegible]

set_password() (rest_framework_simplejwt.models.TokenUser method), 19
 SlidingToken (class in rest_framework_simplejwt.tokens), 21
 TokenObtainPairSerializer (class in rest_framework_simplejwt.serializers), 20
 TokenObtainPairView (class in rest_framework_simplejwt.views), 23
 TokenRefreshSerializer (class in rest_framework_simplejwt.serializers), 20
 TokenRefreshSlidingSerializer (class in rest_framework_simplejwt.serializers), 20
 TokenRefreshSlidingView (class in rest_framework_simplejwt.views), 23
 TokenRefreshView (class in rest_framework_simplejwt.views), 23
 TokenBlacklistSerializer (class in rest_framework_simplejwt.serializers), 20
 TokenBlacklistView (class in rest_framework_simplejwt.views), 23
 TokenObtainPairSerializer (class in rest_framework_simplejwt.serializers), 20
 TokenObtainPairView (class in rest_framework_simplejwt.views), 23
 TokenVerifySerializer (class in rest_framework_simplejwt.serializers), 20
 TokenVerifyView (class in rest_framework_simplejwt.views), 23
 TokenViewSet (class in rest_framework_simplejwt.views), 23
 TokenViewBase (class in rest_framework_simplejwt.views), 23
 TypedToken (class in rest_framework_simplejwt.tokens), 21
 TypedTokenObtainPairSerializer (class in rest_framework_simplejwt.serializers), 20
 TypedTokenObtainPairView (class in rest_framework_simplejwt.views), 23
 TypedTokenRefreshSerializer (class in rest_framework_simplejwt.serializers), 20
 TypedTokenRefreshSlidingSerializer (class in rest_framework_simplejwt.serializers), 20
 TypedTokenRefreshSlidingView (class in rest_framework_simplejwt.views), 23
 TypedTokenRefreshView (class in rest_framework_simplejwt.views), 23
 TypedTokenVerifySerializer (class in rest_framework_simplejwt.serializers), 20
 TypedTokenVerifyView (class in rest_framework_simplejwt.views), 23
 UntypedToken (class in rest_framework_simplejwt.tokens), 22
 user_permissions (rest_framework_simplejwt.models.TokenUser attribute), 19
 username (rest_framework_simplejwt.models.TokenUser attribute), 19
 username_field (rest_framework_simplejwt.serializers.TokenObtainPairSerializer attribute), 20
 validate() (rest_framework_simplejwt.serializers.TokenBlacklistSerializer method), 19
 validate() (rest_framework_simplejwt.serializers.TokenObtainPairSerializer method), 20
 validate() (rest_framework_simplejwt.serializers.TokenObtainSerializer method), 20
 validate() (rest_framework_simplejwt.serializers.TokenObtainSlidingSerializer method), 20
 validate() (rest_framework_simplejwt.serializers.TokenRefreshSerializer method), 20
 validate() (rest_framework_simplejwt.serializers.TokenRefreshSlidingSerializer method), 20
 validate() (rest_framework_simplejwt.serializers.TokenVerifySerializer method), 20
 verify() (rest_framework_simplejwt.tokens.BlacklistMixin method), 21
 verify() (rest_framework_simplejwt.tokens.Token method), 22
 verify_token_type() (rest_framework_simplejwt.tokens.Token method), 22

```
verify_token_type()  
    (rest_framework_simplejwt.tokens.UntypedToken  
     method), 22
```

W

```
www_authenticate_realm  
    (rest_framework_simplejwt.authentication.JWTAuthentication  
     attribute), 18  
www_authenticate_realm  
    (rest_framework_simplejwt.views.TokenViewBase  
     attribute), 23
```